



Point Cloud Processing

Graduate Course INTR-6000P

Week 9 - Lecture 17

Changhao Chen

Assistant Professor

HKUST (GZ)

Toward Multisensory Fusion

Integration with:

- GNSS → absolute positioning
- LiDAR → depth and map consistency
- Wheel encoders → low-speed drift correction

Discussion:

Q1: How to fuse more sensors to estimate states?

Q2: Why is tightly coupled fusion more accurate?



Toward Multisensory Fusion

Q1: How to fuse more sensors to estimate states?

1. Define state vector extension if needed

1. Add extrinsic calibrations, time offsets, additional physical states (e.g., scale, biases, IMU biases already included).

2. Model each sensor's measurement equation

$$\mathbf{z} = h(\mathbf{x}) + \mathbf{n}$$

where

1. $h(\cdot)$ = measurement function (sensor physics)
2. $\mathbf{n}$ = noise model (Gaussian or non-Gaussian)

3. Add sensor factors into estimation

1. **Filtering:** EKF/MSCKF — sensor update step
2. **Optimization / factor graph:** add new residual terms

Toward Multisensory Fusion

Q2: Why is tightly coupled fusion more accurate?

Loose fusion:

"Fuse processed estimates → information discarded → errors propagate"

Tight fusion:

"Fuse raw measurements → retain all information → globally consistent solution"



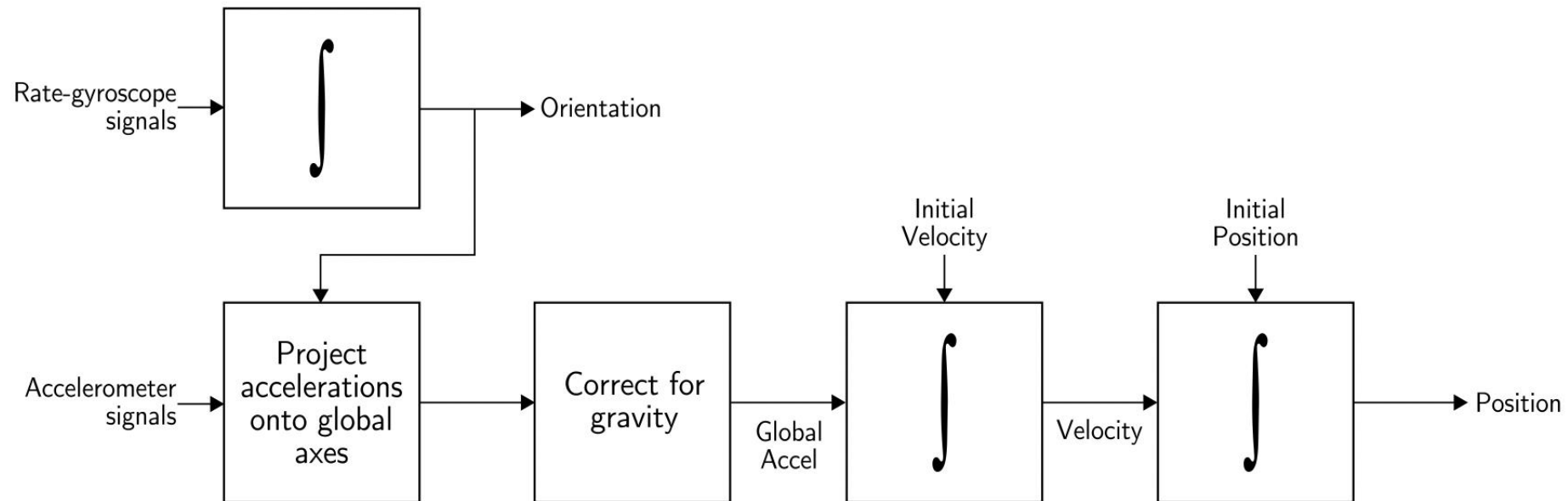
Recap: IMU State Vector

$$\mathbf{x} = [\mathbf{R}, \mathbf{v}, \mathbf{p}, b_g, b_a]$$

Orientation, velocity, position.

Biases evolve as random walks:

$$\dot{b}_a = n_{ba}, \quad \dot{b}_g = n_{bg}$$



Recap: Visual-Inertial Navigation

Pipeline:

Image acquisition → feature extraction/tracking

IMU propagation → predict next state

Visual update → correct drift

Optimization or filtering → refine pose

System State Definition:

$$\mathbf{x} = [\mathbf{R}, \mathbf{v}, \mathbf{p}, b_g, b_a, \text{landmarks}]$$

- Orientation $\mathbf{R}$, velocity $\mathbf{v}$, position $\mathbf{p}$.
- IMU biases b_g, b_a .
- Optional: feature landmarks for visual constraints.

Recap: IMU Preintegration

Preintegrated Quantities

$$\Delta \mathbf{R}_{ij}, \Delta \mathbf{v}_{ij}, \Delta \mathbf{p}_{ij}$$

Accumulate small motion increments:

$$\Delta \mathbf{R}_{ij} = \prod_{k=i}^{j-1} \exp([\omega_k - b_g] \Delta t)$$

Preintegration Equations

$$\Delta \mathbf{R}_{ij} = \mathbf{R}_i^T \mathbf{R}_j$$

$$\Delta \mathbf{v}_{ij} = \mathbf{R}_i^T (\mathbf{v}_j - \mathbf{v}_i - g \Delta t_{ij})$$

$$\Delta \mathbf{p}_{ij} = \mathbf{R}_i^T (\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} g \Delta t_{ij}^2)$$

Preintegration Jacobians and Covariance

$$\mathbf{z}_{IMU} = f(\mathbf{x}_i, \mathbf{x}_j, b_a, b_g) + n$$

Linearize preintegrated terms w.r.t. bias.

Maintain covariance of preintegration errors.

Enables use in optimization as a single factor.

Recap: Fusion Frameworks

1) **Filtering-based:**

sequential estimation (EKF-style).

1) **Optimization-based:**

nonlinear least-squares across time window.

Trade-offs: computational cost vs. accuracy.

Loosely Coupled Fusion:

- Fuse separate **visual odometry** (VO) and **IMU** estimates.
- VO provides relative motion or pose; IMU provides short-term propagation.
- Use EKF or complementary filter for fusion.

Pros: simple, modular.

Cons: less accurate, partial observability.

Tightly Coupled Fusion

- Jointly optimizes IMU and visual feature measurements.
- Single estimation problem with shared state.
- Adds both **visual reprojection** and **IMU preintegration** residuals.

Pros: high accuracy and consistency.

Cons: higher computation.

Point Clouds

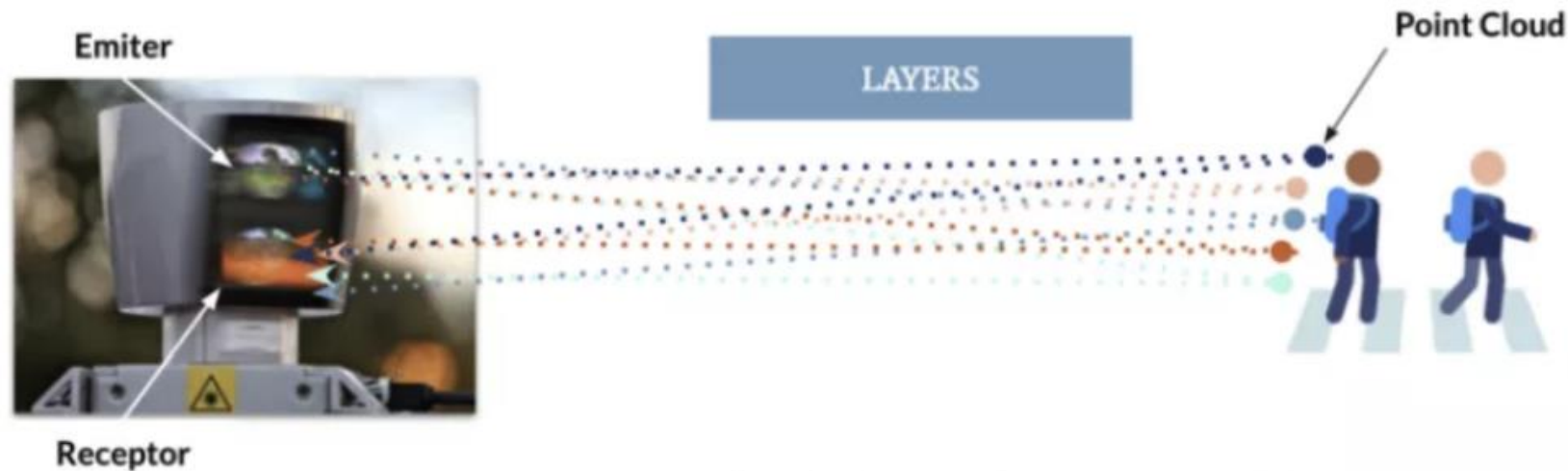
Sources:

- LiDAR sensor (spinning, solid-state, MEMS)
- Depth camera (RGB-D)
- Multi-view stereo

- 3D structure, long range, illumination-independent

Applications

- SLAM, mapping, object detection, scene understanding



LiDAR Sensor

- Time-of-Flight (ToF)
- Scan patterns:
 - Rotating mechanical LiDAR (Hesai, Ouster)
 - Solid-state LiDAR (flash / FMCW / MEMS)
- Key specs:
 - Range, FOV, number of beams, vertical resolution, RPM
 - Accuracy vs distance

Formula: $\text{Distance} = c \times \Delta t / 2$

Rotating Mechanical LiDAR: It performs a 360-degree scan through physical rotation, achieving comprehensive coverage of the surrounding environment and generating a point cloud. It needs to be installed at the highest point of the vehicle to avoid obstructions.

Solid-State LiDAR: It only scans in one direction within a certain angle, which limits its coverage. However, by eliminating the complex high-frequency rotating mechanical structure, its durability is significantly improved, and its size can also be greatly reduced.

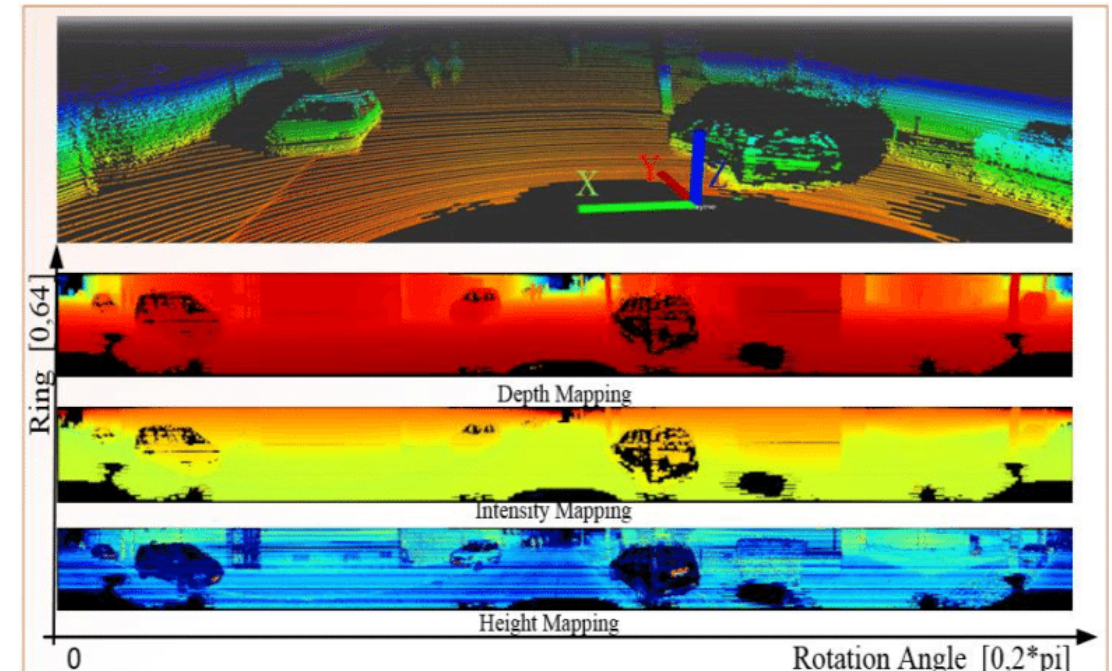


Data Characteristics & Challenges

LiDAR Sensor:

- Its working principle involves the use of multiple laser beams—the more layers, the higher the accuracy. When the laser encounters an obstacle, it is reflected, allowing the precise position (X, Y, Z) of each impact point to be captured, thereby generating a point cloud (X, Y, Z, I).
- The advantage of lasers lies in their ability to focus without significant divergence over long distances.
- However, this also means they cannot bypass obstacles and are highly susceptible to interference—or even complete failure—in adverse weather conditions such as rain, fog, or sandstorms.

- Sparse vs dense point clouds
- Noise & outliers
- Motion distortion
- Occlusion & missing data
- Dynamic environment (vehicles, pedestrians)



Point Cloud Coordinate Systems

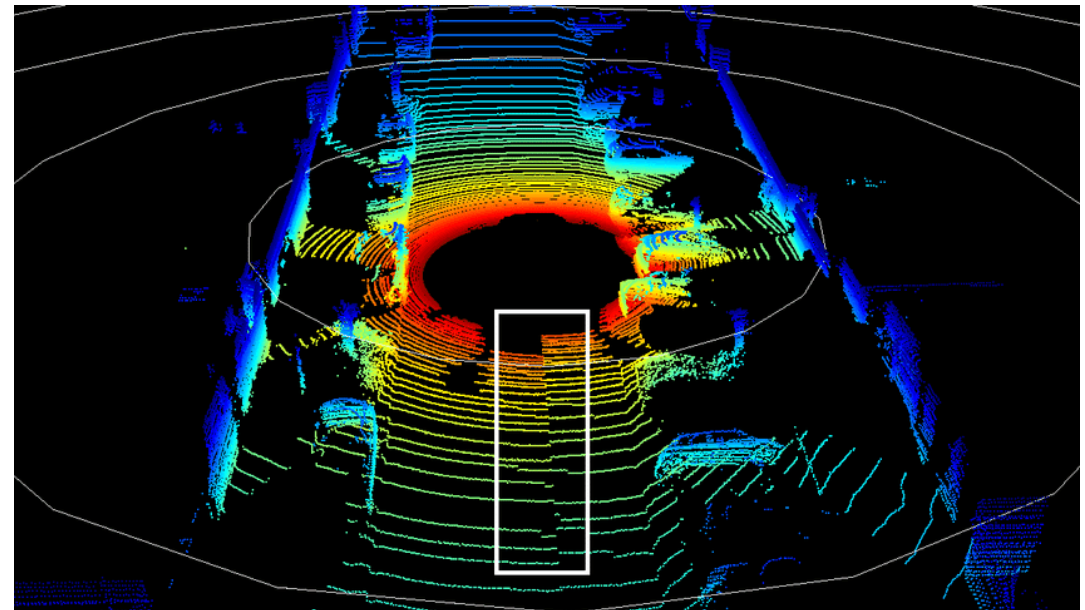
Sensor frame vs vehicle frame vs world frame

SE(3) notation

Extrinsic calibration: LiDAR $\leftrightarrow$ camera $\leftrightarrow$ IMU

Pre-processing Pipeline

- Timestamp & synchronization
- Ego-motion compensation
- Filtering & down-sampling
 - Voxel grid filtering
 - Radius & statistical outlier removal



Point Cloud Registration

- Objective: **find rigid transform $T \in SE(3)$ aligning point sets**
- Error metric: point-to-point vs point-to-plane
- Non-linear least squares (Gauss-Newton, ICP variants)

Align two point clouds $\rightarrow$ estimate relative motion ($SE(3)$)

$$P_j = RP_i + t$$

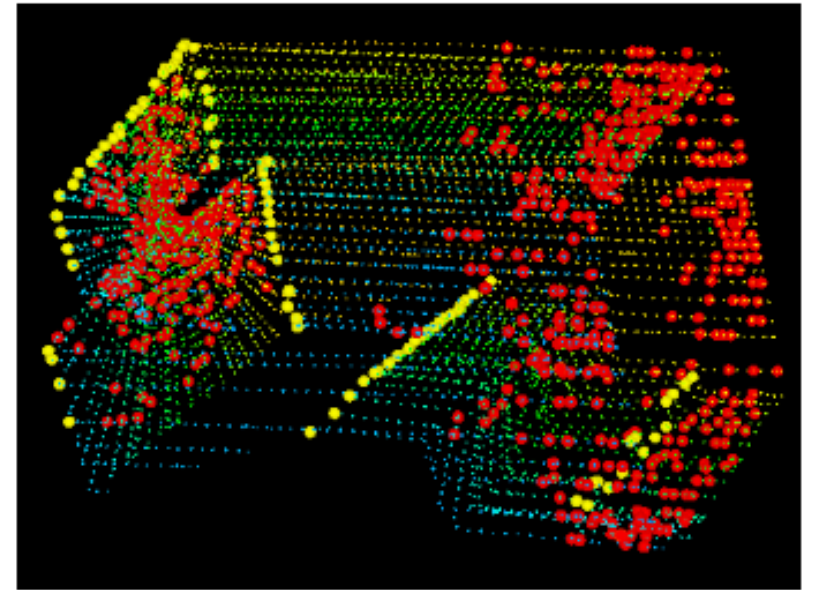
Given:

- Source cloud: $\mathbf{P} = \{p_i\}$
- Target cloud: $\mathbf{Q} = \{q_j\}$

Find transform $\mathbf{T} = [R, t] \in SE(3)$

$$\mathbf{T}^* = \arg \min_{R, t} \sum_i \|Rp_i + t - q_{\pi(i)}\|^2$$

Where $\pi(i)$ is correspondence index.



Types of Registration

Type	Goal	Example
Coarse Global Align	Find big-scale alignment	RANSAC, FPFH
Fine Local Align	Refine small misalign	ICP variants
Real-time Odometry	Incremental motion	LOAM, FAST-LIO2
Semantic-aware	Class features	Semantic-LOAM
Learned Registration	Deep alignment	Deep Closest Point, Predator

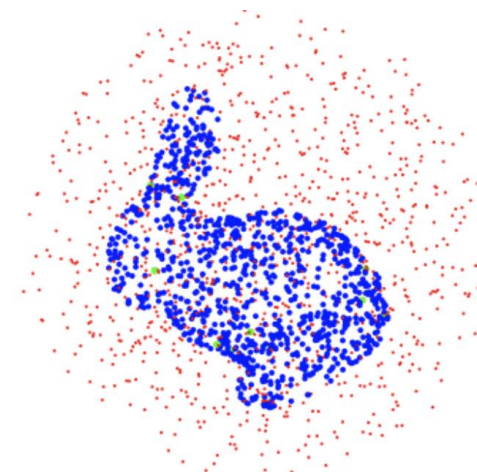
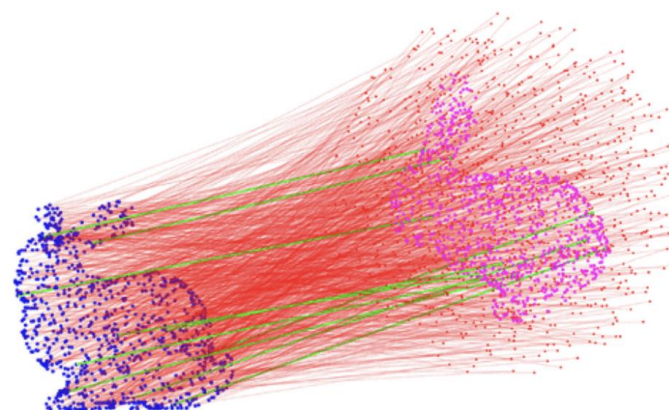
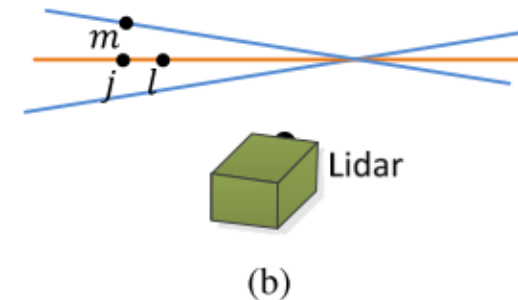
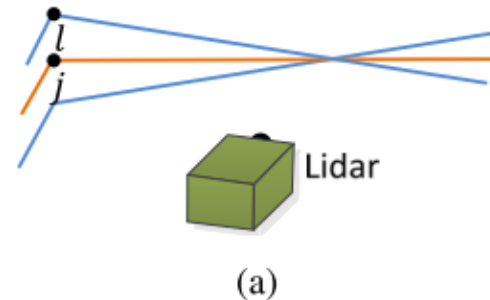
Correspondence Problem

For each point in source, find closest in target.

Strategies:

- Euclidean NN (KD-Tree, FLANN)
- Projective association (camera case)
- Feature matching (FPFH, SHOT, learned descriptors)

Challenge: outliers $\rightarrow$ need robust estimation



ICP Algorithm

Iterative Closest Point

1. Initialize transform guess (IMU/odometry/GNSS)
2. Find correspondences
3. Estimate transform
4. Apply transform
5. Repeat until convergence

Converges if:

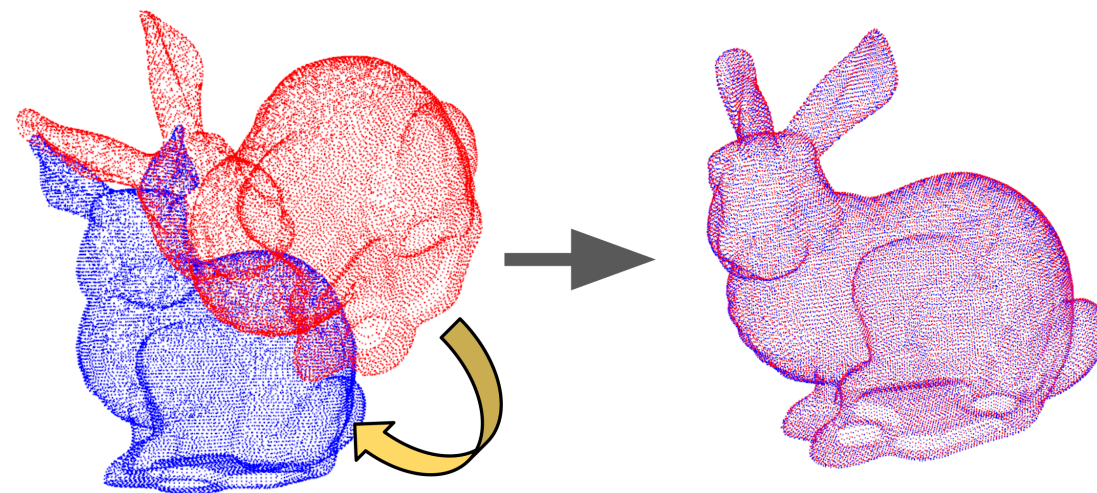
- Good initialization
- Enough overlap
- Noise not too high

Initialization Strategies

Critical for ICP to converge:

- Use odometry / IMU preintegration
- Use NDT or global descriptor (FPFH, scan context)
- Coarse alignment via RANSAC

If you initialize poorly → local minima



ICP Objective

a) Point-to-Point ICP cost

$$E = \sum_i \|Rp_i + t - q_{\pi(i)}\|^2$$

Minimizes the Euclidean distance between corresponding **points**.

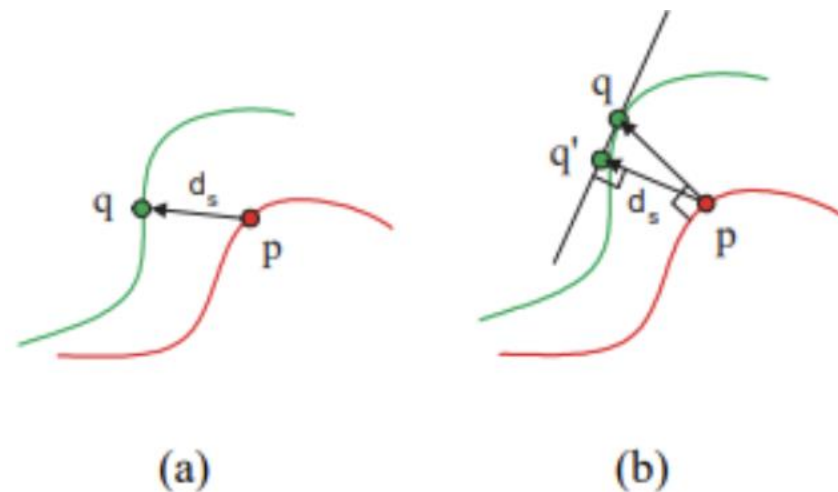
b) Point-to-Plane ICP cost

$$E = \sum_i ((Rp_i + t - q_{\pi(i)}) \cdot n_j)^2$$

Minimizes the distance from the transformed source point to the **tangent plane** of the corresponding target point.

where n_j is target point normal.

Point-to-plane better for LiDAR because surfaces dominate.



Point-to-Plane ICP

Solving for the optimal rotation R and translation T directly in the Point-to-Plane equation is a non-linear problem. To make it tractable, we assume that the angles between the point clouds are small. This allows us to linearize the rotation.

Linearize transform (Gauss-Newton):

$$\delta\xi = (\delta\theta, \delta t)$$

Residual:

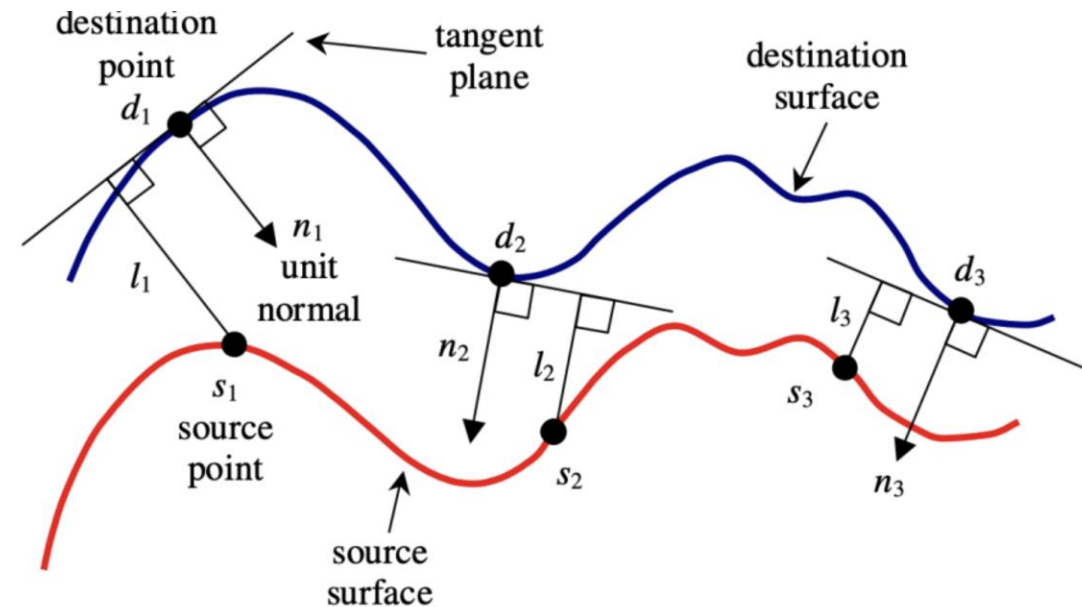
$$r_i = (Rp_i + t - q_{\pi(i)}) \cdot n_i$$

Jacobian:

$$J_i = [(p_i \times n_i)^T \quad n_i^T]$$

Solve normal equation:

$$(J^T J)\delta\xi = -J^T r$$



Generalized ICP (G-ICP)

Interpret each point as Gaussian:

$$C_p, C_q = \text{local covariance}$$

Minimize Mahalanobis:

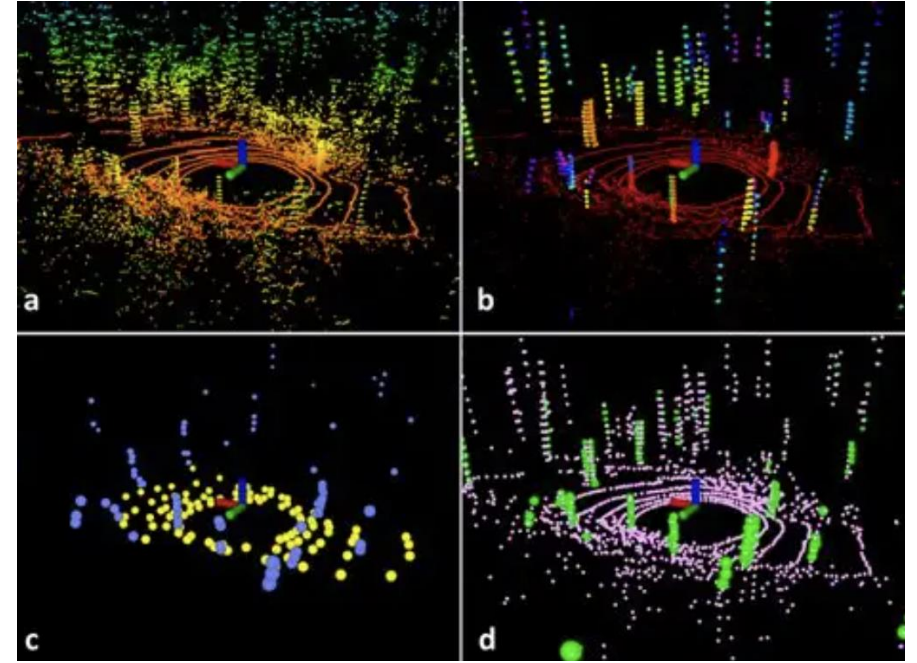
$$E = \sum (Rp_i + t - q_i)^T (C_p + RC_q R^T)^{-1} (Rp_i + t - q_i)$$

Combines point-to-point & point-to-plane as special cases

Point Cloud Feature Extraction

Why Extract Features?

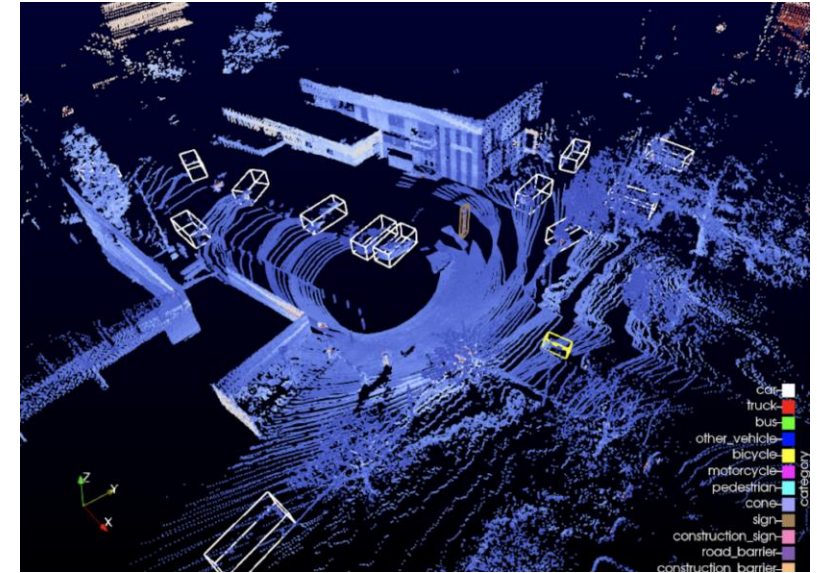
- Raw point clouds are **unordered** and **dense** → inefficient for matching or SLAM.
- Extracting **geometric features** makes registration faster and more robust.



Type	Description	Applications
Keypoints	Distinct, repeatable 3D points (edges, corners)	Registration, loop closure
Edges/Planes	Surface primitives via curvature	LOAM, LIO-SAM
Descriptors	Quantitative shape signatures	Global/local matching
Semantic Features	Learned object-level features	Semantic SLAM, perception

Object Detection in 3D Point Clouds

Detect objects directly in 3D → output **3D bounding boxes** + **class labels**



Approach	Representative Models	Key Idea
Voxel-based	VoxelNet, SECOND, CenterPoint	Discretize into 3D voxels → sparse 3D CNNs
Point-based	PointRCNN, PV-RCNN	Direct point ops → refine proposals
BEV Projection	PointPillars, BEVFusion	Project to Bird's-Eye-View → fast 2D CNN
Fusion (LiDAR + Camera)	PointPainting, TransFusion	Combine geometry + appearance

Object Detection in 3D Point Clouds

Typical Pipeline

1. Preprocess (voxel/BEV/point grouping)
2. Feature extraction (MLP / sparse CNN)
3. Proposal generation (anchor- or center-based)
4. Box regression + classification
5. 3D/BEV NMS

Challenges

- Sparse & irregular LiDAR points
- Long-range distance decay & occlusion
- Real-time requirement for autonomous driving
- Multi-sensor alignment (when fused with camera/IMU)

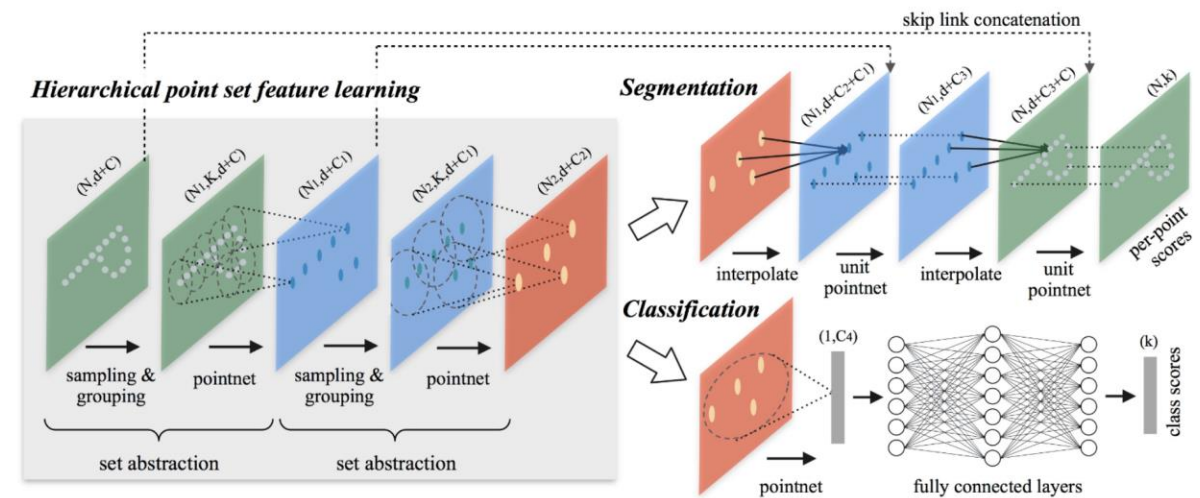
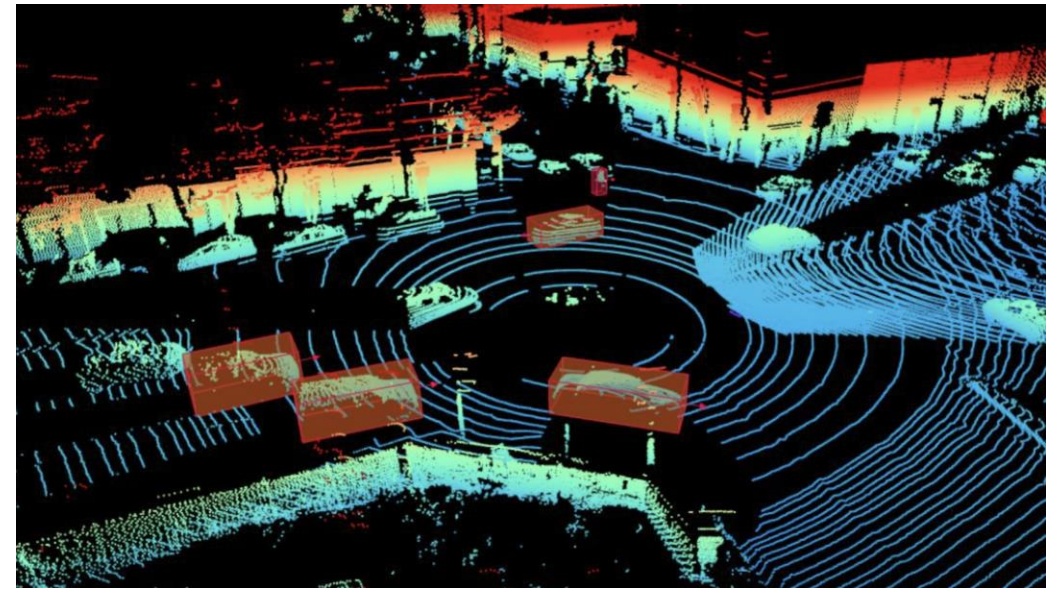


Figure 1. PointNet++ Architecture for Point Set Segmentation and Classification.

Mapping & Dense Reconstruction

Build a **consistent, dense 3D map** from sequential LiDAR/visual scans for navigation & scene understanding.

Pipeline

1. Pose estimation (SLAM / VO)
2. Point cloud registration (ICP / feature fusion)
3. Map integration (Octree/TSDF/ESDF)
4. Loop closure & global BA pose graph
5. Surface extraction (Marching Cubes / mesh refinement)

Method	Description	Examples
Point Cloud Fusion	Accumulate registered scans	LOAM, LIO-SAM
Voxel Mapping	Discretize space → store occupancy	OctoMap, Voxblox
TSDF Volume Fusion	Average signed distance fields → smooth mesh	KinectFusion, Voxblox, NVBlox
ESDF / SDF Maps	Signed distance for planning	FIESTA, nvblox
Neural Implicit Mapping	MLP learns scene geometry	NeRF-SLAM, NICE-SLAM



Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)